Class 30: Model Deployment and Prediction Serving

Session 30 of 36 – MLOps Block

Data-driven Systems Engineering

Agenda

- From a trained artifact to a callable, running service.
- Flask as a service layer, Streamlit as a presentation layer.
- The serving components of this repository, and what they lack.
- Serving patterns and production concerns beyond a demonstration.
- Closing the Data-Driven block and opening the MLOps block.

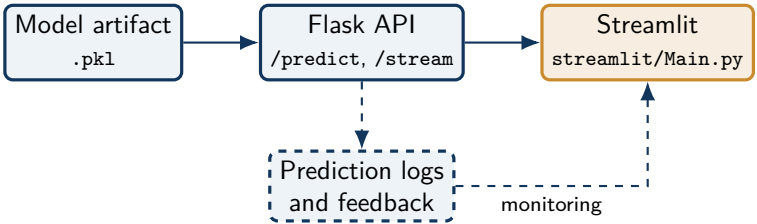
A model creates value only through an interface

- A trained model that cannot be invoked or interpreted has limited practical value, regardless of its offline metrics.
- An interface makes predictions available to users, services, and operational workflows.
- A served model is a small software system with three parts: an interface, a preprocessing contract, and an operational surface (health, logs, versions).

Flask and Streamlit play different roles

Flask	A web framework for programmable endpoints over the Hypertext Transfer Protocol (HTTP), such as `POST /predict` or a streaming route.
Streamlit	A framework for interactive user interfaces in Python: demonstrations, dashboards, exploratory tools.
Combined	A common pattern: Flask as the back-end service, consumed by other programs, and Streamlit as the human-facing front end.

Serving components of this repository



- Solid elements exist in the repository; dashed elements do not yet exist: no service records its predictions, so nothing can be monitored after deployment.
- Closing this gap is the subject of continuous monitoring (Class 32).

Repository example: the fraud scoring endpoint

```python
model = pickle.load(open("notebooks/best_random_forest_model.pkl", "rb"))

@app.route("/")
def index():
    return jsonify({"message": "Model API running"}), 200

@app.route("/predict", methods=["POST"])
def predict():
    data = request.get_json(force=True)
    features = np.array(data["features"]).reshape(1, -1)
    pred = model.predict(features)[0]
    return jsonify({"prediction": int(pred)})
```

- This is the complete serving logic of `Class6_model_api.py`: load the model once at start-up and expose one prediction route.
- It is intentionally minimal, so that what a service boundary contains, and what it still lacks, is visible.

What the minimal endpoint lacks

- **A deployable artifact:** the file `notebooks/best_random_forest_model.pkl` does not exist in the repository, so the service fails at start-up; the training notebook saves `model.pkl`.
- **The preprocessing contract:** the model was trained on standardized features, but the endpoint passes raw features without the scaler. This *training–serving skew*, $f(\mathbf{x}) \neq f(\phi_{\text{train}}(\mathbf{x}))$, produces wrong predictions without any error message.
- **Input validation:** `data["features"]` is trusted to contain exactly 30 numbers; any other request raises an unhandled exception.
- **A useful response:** only a bare class $0/1$, without the probability, the threshold, or the model version.
- **Operational surface:** no logging, and `debug=True` on `0.0.0.0` exposes the interactive debugger to the network, which allows remote code execution.

Training–serving skew, measured

The deployed random forest (`streamlit/model.pkl`) applied to the 56,962 test transactions, with and without the scaler used in training ($\tau = 0.5$):

Input to the model	Frauds detected (TP)	Missed (FN)	False alarms (FP)
Standardized, as in training	81	17	5
Raw features, as in `Class6_model_api.py`	79	19	16

- The service raises no error: it silently returns worse predictions, with three times as many false alarms.
- The damage is moderate here because `V1`–`V28` are principal components whose scale is already close to unit variance; only `Time` and `Amount` change substantially. With features on very different scales, the degradation would be much larger.
- Remedy: persist the preprocessing together with the model (or as one `Pipeline` object) and load both in the service.

A corrected service: `serving/fraud_api.py`

	`Class6_model_api.py`	`serving/fraud_api.py`
Artifact	Missing file; the service does not start	`streamlit/model.pkl`, identified by a hash (model version)
Preprocessing	None (training–serving skew)	The scaler fitted in training
Validation	None; malformed input raises an exception	30 finite numbers, otherwise status 400 with a message
Response	Bare class $0/1$	Probability, explicit threshold, decision, version, request identifier, latency
Logging	None	One JSON line per prediction (inputs and outputs)
Operations	`debug=True` on `0.0.0.0`	`GET /health`; debugger disabled; bound to localhost

The minimal file is kept unchanged in the repository as a teaching example. Run the corrected service from the repository root: `python serving/fraud_api.py` (port 5001).

The corrected service: real responses

`POST /predict`, a fraudulent test transaction (200)

```
{
  "fraud_probability": 0.915,
  "is_fraud": true,
  "threshold": 0.5,
  "model_version": "1382290f145e",
  "request_id": "f98e4f26-...",
  "latency_ms": 11.21
}
```

29 values instead of 30 (400)

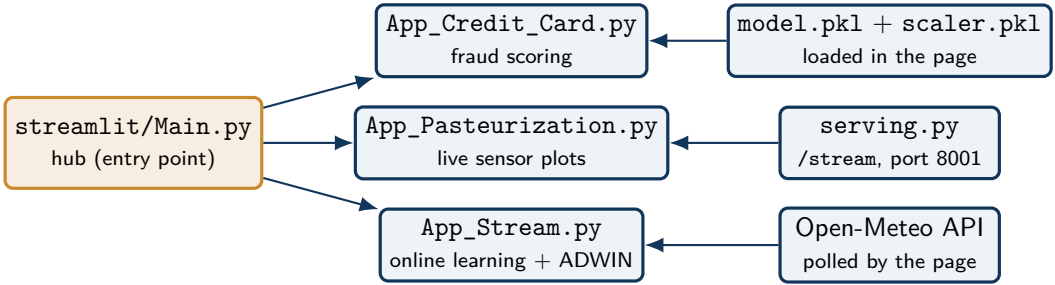
```
{"error": "\"features\" must be a
 list of 30 numbers (Time, V1,
 ..., Amount)"}
```

A non-numeric value (400)

```
{"error": "\"features\" must contain
 only finite numbers"}
```

- A legitimate transaction from the same test split receives a probability of 0.005. Every response is also appended to `logs/predictions.jsonl`, which is excluded from Git.

Repository example: the dashboard hub



- `streamlit run streamlit/Main.py` starts the hub; Streamlit discovers every file in `pages/` and lists it in the navigation.
- The three pages use three different serving patterns: a model loaded inside the user interface, a separate streaming service, and an external application programming interface (API).

Serving patterns

Batch scoring	Predictions are computed periodically for a stored dataset and stored for later use; latency is irrelevant, throughput matters.
Online request–response	A client sends one request and waits synchronously for one prediction, as in `Class6_model_api.py`; latency is critical.
Streaming	Predictions are computed continuously over an incoming event stream, as in the pasteurization case (Classes 18–19).

Production concerns not covered by the demonstrations

- **Performance:** latency budgets and load tests. Measured on 1,000 sequential requests to `serving/fraud_api.py`: 50th percentile 14 ms, 95th percentile 17.5 ms. With the random forest as trained (`n_jobs=-1`), the same requests took 41 and 86 ms: parallelism designed for training slows down single predictions.
- **Serving infrastructure:** Flask itself warns that its development server must not be used in production; a production deployment uses a Web Server Gateway Interface (WSGI) server (e.g. Gunicorn) behind a reverse proxy.
- **Versioning and compatibility:** `model.pkl` was serialized with scikit-learn 1.7.2, whereas `requirements.txt` pins 1.6.0; a model registry stores the artifact with the exact library versions needed to load it.
- **Observability:** structured logs (now present), request tracing, and alerts on error rates.

From a demonstration service to a production deployment



- Each step is developed in the MLOps block: containers and continuous integration and delivery (CI/CD) in Class 31, monitoring and scalability in Class 32.

Repository connections

Repo activity

- `Class6_model_api.py`: the minimal Flask serving example for the fraud case; `serving/fraud_api.py`: its corrected counterpart.
- `streamlit/Main.py`: the hub that presents the fraud, pasteurization, and online-learning pages together.
- `pasteurization/serving.py`: the same serving concerns applied to a continuous stream instead of single requests.

Readiness checklist before calling a model “served”

- Does the service start from a clean checkout of the repository, with the referenced artifacts present?
- Can it reject a malformed request with an informative error instead of crashing?
- Is the preprocessing at inference identical to the preprocessing used in training?
- Is it recorded which model version answered each request?
- Could a teammate other than the author redeploy the service from its documentation?

Papers and materials

Papers and materials

- Sculley, D., Holt, G., Golovin, D., Davydov, E., Phillips, T., Ebner, D., Chaudhary, V., Young, M., Crespo, J.-F., Dennison, D. (2015). Hidden technical debt in machine learning systems. *Advances in Neural Information Processing Systems*, 28, 2503–2511.
- Breck, E., Cai, S., Nielsen, E., Salib, M., Sculley, D. (2017). The ML test score: A rubric for ML production readiness and technical debt reduction. *Proceedings of the IEEE International Conference on Big Data*, 1123–1132.
- Paleyes, A., Urma, R.-G., Lawrence, N. D. (2022). Challenges in deploying machine learning: A survey of case studies. *ACM Computing Surveys*, 55(6), 114:1–114:29.
- Gift, N., Deza, A. (2021). *Practical MLOps*. O'Reilly.
- Wiggins, A. (2017). The Twelve-Factor App.
- Documentation: Flask; Streamlit.

Take-away message

A served model is a software system, not a `.pkl` file:

it needs its exact preprocessing, validated inputs,
informative responses, and a record of every prediction.

Thank you for your attention.

Next: Class 31 — good practices with Docker and CI/CD.